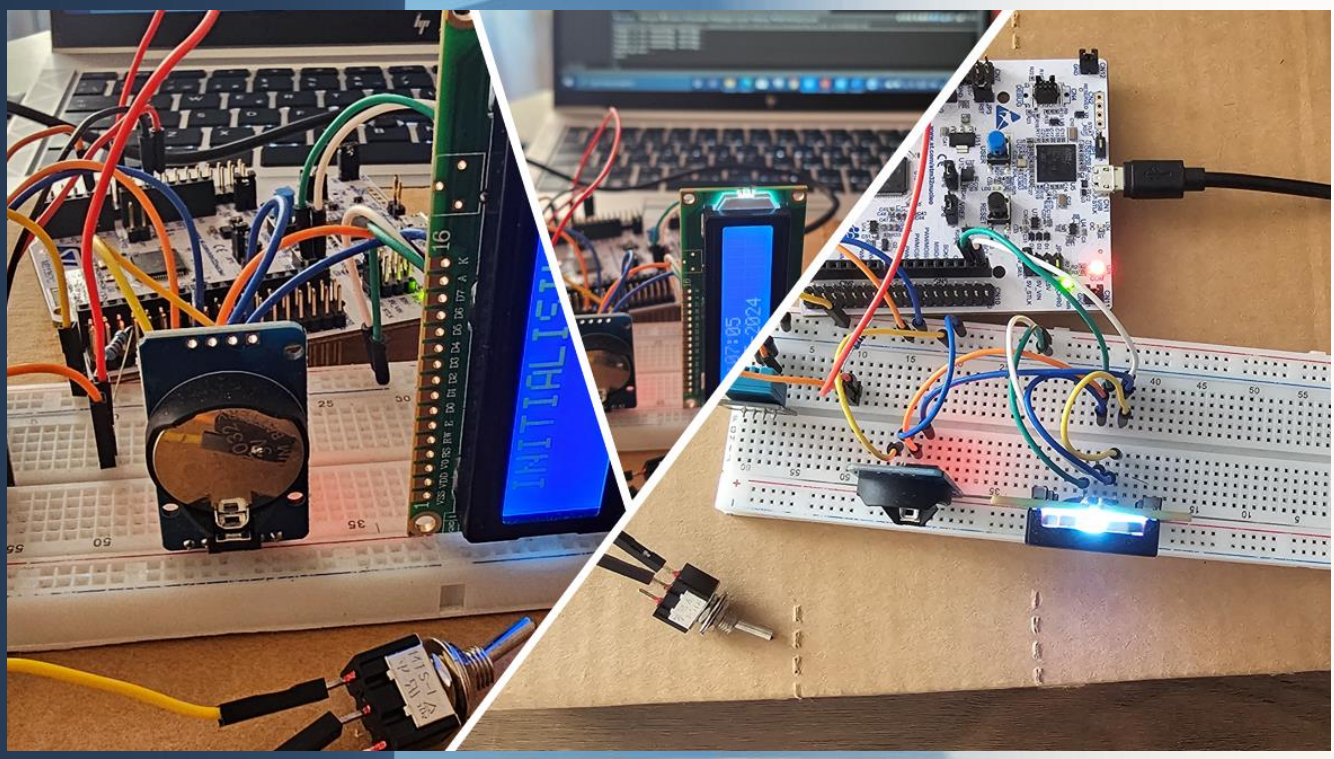
UNIVERSITÀ DEGLI STUDI DI PALERMO

Sensor and signal conditioning for digital measurement

STM32 Nucleo Embedded System Clock and Humidity/Temperature



CHAHINEZ ABDELLAOUI

USED HARDWARE:

Sensors Used:

DHT11
Datasheet Link: DHT11.PDF

RTC Used:

DS3231
Datasheet Link: DS3231.PDF

Board Used:

STM 32 NUCLEO G474RE
Datasheet Link: DS12288.PDF
NUCLEO-G474RE | MBED

LCD Used:

EONE LCD GJD 1602IIC
Datasheet Link: LCD1602_I2C_MODULE_DATASHEET (WAVESHARE.COM)

USED SOFTWARE:

STM32 CUBEMX
STM32 CUBE IDE

Description of the project idea:

In this project, we will build a temperature and humidity monitoring system using a microcontroller, temperature and humidity sensor, real-time clock (RTC) module, LCD display, and a switch for user interaction.

The system will continuously measure temperature and humidity from the environment, then display the current time, temperature, and humidity on an LCD screen, and allow the user to toggle between displaying temperature/ humidity and time using a switch.

Components:

Microcontroller (e.g., Arduino, STM32 Nucleo)
Temperature and Humidity Sensor (e.g., DHT11, DHT22)
Real-Time Clock (RTC) Module (e.g., DS3231)
LCD Display (16x2 or similar)
Switch (toggle or push-button)
Breadboard and jumper wires
Power supply (battery or USB)

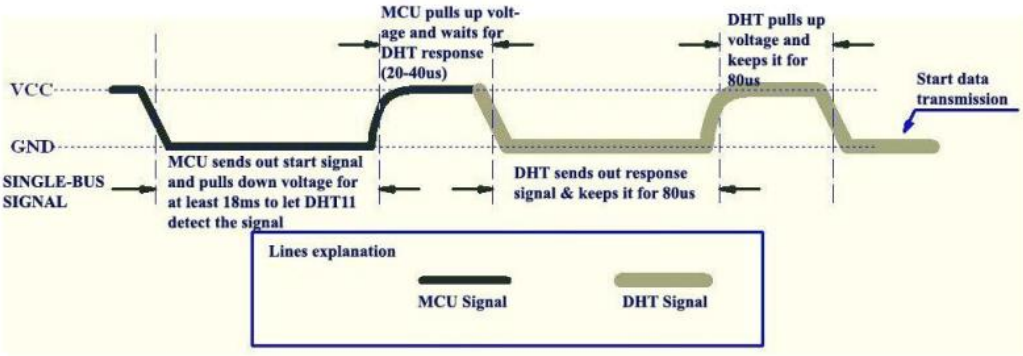
Working principle:

Sensor:

simplified explanation of how the DHT11 sensor works:
Humidity Measurement: The humidity-sensitive capacitor changes its capacitance based on the surrounding humidity. As the humidity increases, the capacitance changes accordingly.
Temperature Measurement: The thermistor (NTC-TYPE) changes its resistance with variations in temperature. As the temperature increases, the resistance of the thermistor decreases, and vice versa.
Signal Processing: The DHT11 sensor contains internal circuitry to convert the changes in capacitance (humidity) and resistance (temperature) into digital signals.
Digital Output: The DHT11 sensor provides a digital signal output using a single-wire protocol. It sends a series of digital pulses representing the measured humidity and temperature values. These pulses are then interpreted by a microcontroller or other digital device.

Steps to read data from the sensor and display it on the LCD:

- **Initialization:** To initiate communication with the DHT11 sensor, the microcontroller sends a start signal to the sensor.



As shown above, in order to initialize the sensor, we have to first pull the data line LOW for around 18 ms.
After this DHT11 will pull the line LOW for 80 us, and than HIGH for 80 us.
If the conditions are satisfied, that means the sensor is present, and we can proceed with reading the data.

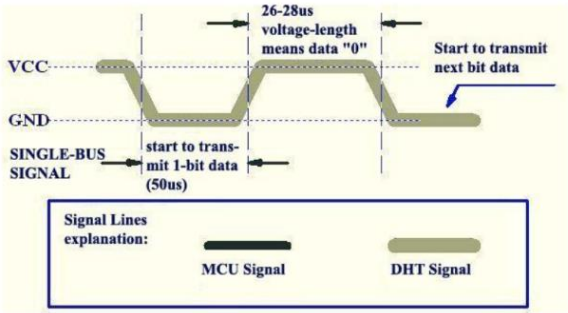


Figure 4 Data "0" Indication

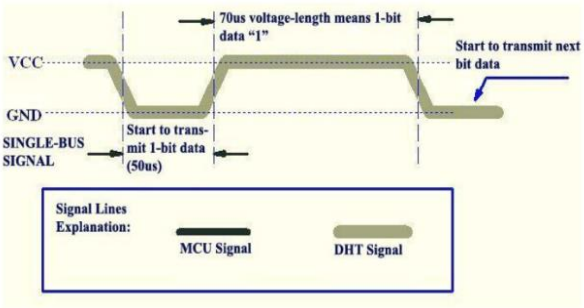


Figure 5 Data "1" Indication

Now DHT11 will send 40 bits of data. Each bit's transmission begins with low-voltage-level that last 50 us, the following high-voltage-level signal's length decides whether the bit is "1" or "0".
If the length of high-voltage-level is around 26-28 us, the bit is "0".
And if the length is around 70 us, then the bit is "1".
The 40 bits sent by DHT11 are as follows:

DATA = 8-bit integral RH data + 8-bit decimal RH data + 8-bit integral T data+8 bit decimal T data + 8-bit checksum

Once this is done, the sensor will be initialized and start transmitting.

- **Data Request:** After sending the start signal, the microcontroller waits for the sensor to respond. Then, it sends a data request signal to the sensor.

- **Data Transmission:** The DHT11 sensor then responds by sending data to the microcontroller. The data typically includes temperature and humidity readings, encoded in a specific format.

- **Data Interpretation:** Once the microcontroller receives the data from the sensor, it interprets the data and extracts the temperature and humidity readings.

- **Error Checking:** The communication protocol of the DHT11 sensor include error-checking mechanisms to ensure the integrity of the data. The sensor uses a checksum to verify the correctness of the received data.
  If the data transmission is right, checksum should be the last 8 bit of "8 bit integral RH data+8 bit decimal RH data+8 bit integral T data+8 bit decimal T data"

- **Repeat:** The microcontroller can periodically request data from the sensor to continuously monitor temperature and humidity.

Real Time Clock:

simplified explanation of how the DS3231 RTC works:

Real time clock with an integrated temperature compensated 32KHz crystal oscillator (TCXO) and crystal. The TCXO provides a stable and accurate reference clock, and maintains the RTC to within ±2 minutes per year accuracy from -40°C to +85°C. T. The device incorporates a battery input and maintains accurate timekeeping when main power to the device is interrupted. It is offered in a 16-pin, 300-mil SO package.
The DS3231 Real-Time Clock (RTC) communicates using the Inter-Integrated Circuit (I2C) protocol and RTC address is 0xD0. (I2C is a widely used serial communication protocol that allows multiple devices to communicate with each other over a two-wire interface).

The communication with the DS3231 RTC typically works by tracking the following steps:

- **Initialization:** The microcontroller initiates communication by sending a start condition on the I2C bus.

- **Addressing**: The microcontroller sends the 7-bit slave address of the DS3231 RTC device followed by a read/write bit indicating whether it wants to read from or write to the device.

- **Data Transfer**: Depending on whether the microcontroller wants to read or write data, it sends or receives data bytes to or from the DS3231 RTC.

- **Acknowledgment:** After receiving each byte of data, the DS3231 RTC sends an acknowledgment signal (ACK) or a not-acknowledgment signal (NACK) back to the microcontroller to indicate whether it successfully received the data.

- **In the code,** in the HAL_I2C_Mem_Read and HAL_I2C_Mem_Write functions, the ACK signal is automatically generated by the I2C hardware peripheral after each byte of data is received or transmitted. The HAL library handles the ACK/NACK signalling internally.

- **Stop Condition:** Once the data transfer is complete, the microcontroller sends a stop condition on the I2C bus to end the communication.
  The stop condition is generated by the HAL_I2C_Mem_Read and HAL_I2C_Mem_Write functions when the data transfer is complete.

After reading or writing the specified number of bytes, the HAL library automatically generates the stop condition to end the I2C communication transaction.

The following is the picture from the datasheet of the device. It shows the registers available in the DS3231 for writing and reading data.
we are only going to interface the RTC part. therefore, we are interested in the registers ranging from address 00h to 06h for the clock and date.

Figure 1. Timekeeing Registers

ADDRESS	BIT 7 MSB	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0 LSB	FUNCTION	RANGE
00H	0	10 Seconds			Seconds				Seconds	00–59
01H	0	10 Minutes			Minutes				Minutes	00–59
02H	0	12/$\overline{24}$	$\overline{AM}$/PM 10 Hour	10 Hour	Hour				Hours	1–12 + $\overline{AM}$/PM 00–23
03H	0	0	0	0	0	Day			Day	1–7
04H	0	0	10 Date		Date				Date	00–31
05H	Century	0	0	10 Month	Month				Month/ Century	01–12 + Century
06H	10 Year				Year				Year	00–99
07H	A1M1	10 Seconds			Seconds				Alarm 1 Seconds	00–59
08H	A1M2	10 Minutes			Minutes				Alarm 1 Minutes	00–59
09H	A1M3	12/$\overline{24}$	$\overline{AM}$/PM 10 Hour	10 Hour	Hour				Alarm 1 Hours	1–12 + $\overline{AM}$/PM 00–23
0AH	A1M4	DY/$\overline{DT}$	10 Date		Day				Alarm 1 Day	1–7
					Date				Alarm 1 Date	1–31
0BH	A2M2	10 Minutes			Minutes				Alarm 2 Minutes	00–59
0CH	A2M3	12/$\overline{24}$	$\overline{AM}$/PM 10 Hour	10 Hour	Hour				Alarm 2 Hours	1–12 + $\overline{AM}$/PM 00–23
0DH	A2M4	DY/$\overline{DT}$	10 Date		Day				Alarm 2 Day	1–7
					Date				Alarm 2 Date	1–31
0EH	$\overline{EOSC}$	BBSQW	CONV	RS2	RS1	INTCN	A2IE	A1IE	Control	—
0FH	OSF	0	0	0	EN32kHz	BSY	A2F	A1F	Control/Status	—
10H	SIGN	DATA	DATA	DATA	DATA	DATA	DATA	DATA	Aging Offset	—
11H	SIGN	DATA	DATA	DATA	DATA	DATA	DATA	DATA	MSB of Temp	—
12H	DATA	DATA	0	0	0	0	0	0	LSB of Temp	—

<u>**Display**</u>:

The LEQUID_CRYSTAL_DISPLAY (LCD) will show the current time/date and temperature/humidity.
The LCD communicates using the Inter-Integrated Circuit (I2C) protocol and its address is 0x7E. Depending on the user's choice It will be divided into two sections: either the top section will display the time and the bottom section will display date or the top section will display the temperature and the bottom section will display humidity.

communication with an I2C-enabled LCD:

- **Initialization**:
  Initialization of the I2C communication interface on the microcontroller.
  Initialization of the LCD module by sending appropriate initialization commands over the I2C bus. This typically includes configuring the display settings, such as number of rows and columns, backlight control, etc.

- **Sending Commands and Data**:
  To send commands or data to the LCD, send specific byte sequences over the I2C bus.
  To display characters or strings, send data bytes representing the characters' ASCII codes.

- **Handling ACK/NACK Signals**:
  After sending each byte of data or command to the LCD, wait for an acknowledgment (ACK) or not-acknowledgment (NACK) signal from the LCD.
  If the LCD successfully receives the byte, it sends an ACK signal. If there's an error or the LCD cannot process the data, it sends a NACK signal. All those signals are of an internal type.

- **Displaying Content**:
  After initializing the LCD and sending necessary commands and data, start displaying content on the LCD.

- **Updating Content**:
  To update the content displayed on the LCD, send new commands and data over the I2C bus.
  periodically as needed.

User Interaction:

The switch will allow the user to toggle between displaying time/date and temperature/humidity on the LCD screen. Each time the switch is pressed, the display will switch between showing time/date and temperature/humidity.

Power Management:

The system will be powered by a USB power supply by Connecting an STM Nucleo development board to a PC.

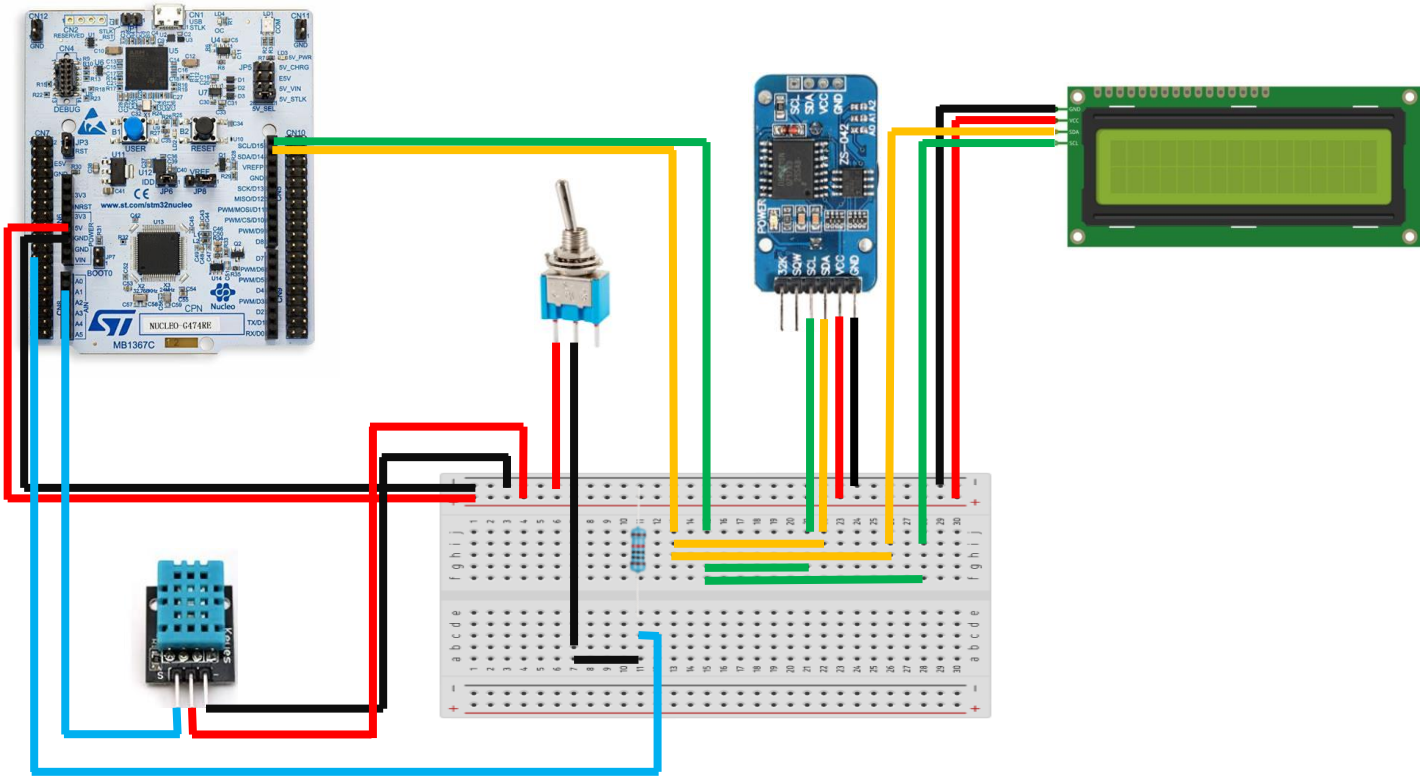
Project Implementation Steps:

- Connect the temperature and humidity sensor, RTC module, LCD display, and switch to the microcontroller according to their respective pinouts.
- Write the code to initialize the sensor, RTC module, LCD display, and configure the switch for input.
- Implement functions to read temperature and humidity from the sensor, read time from the RTC module, and display the values on the LCD screen.
- Implement functionality to toggle between displaying temperature and humidity based on the switch input.
- Test the system by monitoring temperature and humidity readings on the LCD display and verifying the accuracy of the displayed time.
- Fine-tune the system as needed and ensure proper functioning in different environmental conditions.

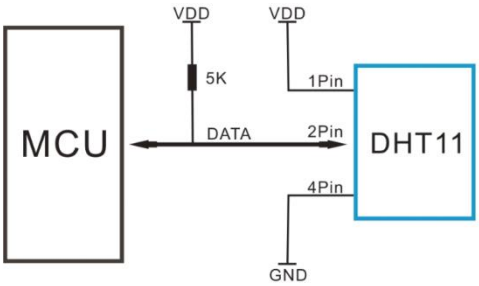
Expected Outcome:

Upon completion, the project will result in a functional temperature and humidity monitoring system with real-time clock display. Users will be able to view the current time, date, temperature, and humidity on an LCD screen and switch between displaying time/date and temperature/humidity as desired. The system will provide a practical demonstration of sensor interfacing, signal conditioning, user interaction, and real-time clock functionality in a digital measurement context.

Wiring diagram and signal conditioning:



The Sensor:



the purpose of the pull-up (resistor connected between a signal line and a positive voltage) resistor is to ensure proper signal integrity and noise immunity in the communication between the DHT11 sensor and the microcontroller.

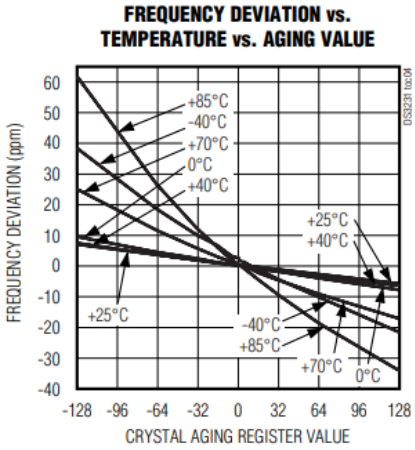
The choice of resistor value depends on factors such as cable length and the capacitance of the cable, and it helps to stabilize the signal levels and prevent signal degradation over longer distances.

THE SWITCH:

Pull-up or Pull-down Resistors: In some cases, external pull-up or pull-down resistors may be used with switches to ensure a defined logic level when the switch is open. This helps prevent floating inputs and ensures reliable signal detection by the connected digital device.

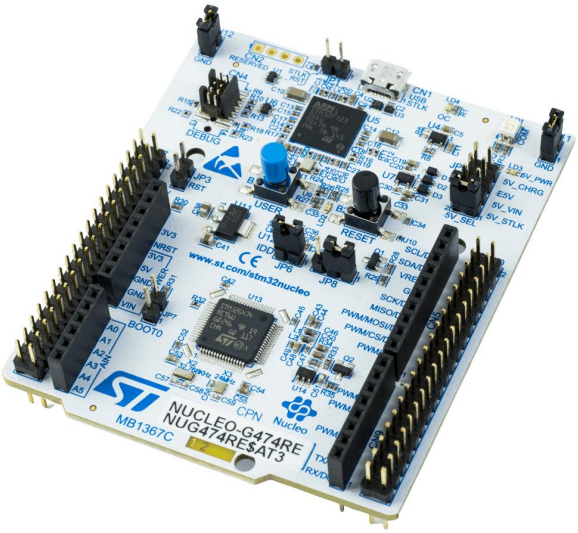
The Real Time Clock:

The DS3231 RTC includes a built-in temperature-compensated crystal oscillator (TCXO) that adjusts the frequency of the oscillator circuit in response to changes in temperature. This temperature compensation mechanism ensures that the clock's timekeeping accuracy remains stable in environments with fluctuating temperatures and over a wide range of temperatures.



Description of used hardware:

Microcontroller: STM32 nucleo G474RE:



- **Powerful Cortex-M4 Core**: The ST Nucleo G474RE is powered by an Arm Cortex-M4 processor, providing sufficient processing power to handle real-time clock operations, sensor data processing, LCD interfacing, and overall system control.

- **Rich Peripheral Set**: The microcontroller comes with a variety of built-in peripherals, including timers, communication interfaces (such as I2C), GPIOs, and hardware timers. These peripherals are essential for interfacing with an RTC (Real-Time Clock), an LCD display, and a sensor.

- **Integrated Development Environment (IDE):** ST provides a comprehensive and user-friendly development environment, like STM32CubeIDE, which supports the ST Nucleo G474RE. This makes it easier for developers to write, debug, and test code for the microcontroller.

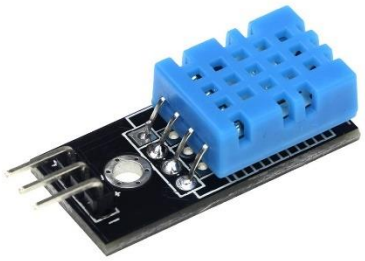
- **CMSIS Standard Support**: The ST Nucleo G474RE supports the **Common Microcontroller Software Interface Standard (CMSIS)**. CMSIS provides a standardized hardware abstraction layer for Cortex-M processors, making it easier to write portable and reusable code.

- **Low Power Features:** For projects that involve a real-time clock and sensor, power efficiency is crucial. The microcontroller offers low-power modes and features that help optimize energy consumption when the system is in idle states.

- **Cost-Effective Solution:** The ST Nucleo G474RE is a cost-effective solution for embedded systems development. It provides a balance between performance and cost, making it suitable for various applications.

Sensor DHT11:

- **Ease of Use:** It provides a simple digital output that can be directly interfaced with microcontrollers or single-board computers without the need for complex circuitry or calibration.



- **Integrated Sensor:** The DHT11 sensor combines both humidity and temperature sensing capabilities in a single compact package. This integrated design simplifies the wiring and installation process, making it convenient for applications where space is limited, or multiple sensors are required.

- **Decent Accuracy for Basic Applications**: it provides decent accuracy for many basic applications. It is suitable for applications where precise humidity and temperature measurements are not critical, such as home automation, weather monitoring, and environmental sensing.

Item	Measurement Range	Humidity Accuracy	Temperature Accuracy	Resolution
DHT11	20-90%RH 0-50 ℃	±5%RH	±2℃	1

- **Wide Availability:** The DHT11 sensor is widely available from various manufacturers and distributors, making it easy to source components for theproject. It is compatible with popular development platforms and programming languages, allowing for seamless integration into the existing hardware and software setup.

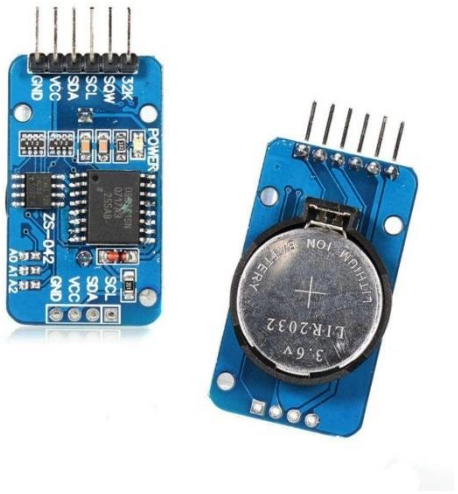
- Low P**ower Consumption:** The DHT11 sensor typically operates at low power, making it suitable for battery-powered or energy-efficient applications. It consumes minimal power during standby mode and only activates when a measurement is requested, helping to conserve energy and prolong battery life.

- **Cost-Effectiveness:** The DHT11 sensor is generally more affordable compared to other humidity and temperature sensors available in the market. If cost is a significant factor in the project, the DHT11 sensor can be a budget-friendly option.

RTC (real time clock): DS3231

Features

- Accuracy ±2ppm from 0°C to +40°C
- Accuracy ±3.5ppm from -40°C to +85°C
- Battery Backup Input for Continuous Timekeeping
- Operating Temperature Ranges
  Commercial: 0°C to +70°C
  Industrial: -40°C to +85°C
- Low-Power Consumption
- Real-Time Clock Counts Seconds, Minutes, Hours, Day, Date, Month, and Year with Leap Year Compensation Valid Up to 2100
- Two Time-of-Day Alarms
- Programmable Square-Wave Output
- Fast (400kHz) I²C Interface
- 3.3V Operation
- Digital Temp Sensor Output: ±3°C Accuracy
- Register for Aging Trim
- $\overline{\text{RST}}$ Input/Output
- UL Recognized

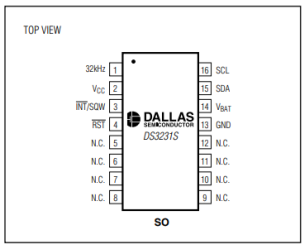


- **Accurate Timekeeping:** RTC maintains accurate timekeeping when main power to the device is interrupted. The DS3231 is a serial RTC driven by a Temperature compensated 32kHz crystal oscillator. The TCXO provides a stable and accurate reference clock and maintains the RTC to within ±2 minutes per year accuracy from -40°C to +85°C (the long-term accuracy).

- **Low Power Consumption**: RTC typically have low power consumption, making them suitable for battery-operated devices or applications where energy efficiency is a concern.

- **I2C Interface**: RTCs communicate with microcontrollers using standard communication interfaces I2C, making them easy to integrate into various microcontroller-based projects.

Pin Configuration



- **Integrated Alarms**: this RTCs come with integrated alarms that can be configured to trigger specific actions or wake up the microcontroller at predefined times. This feature can be useful for scheduling tasks or events in application.

- **Good register implementation:** This allows the smooth communication with the device and higher readability of the vcode.

Figure 1. Timekeeing Registers

ADDRESS	BIT 7 MSB	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0 LSB	FUNCTION	RANGE
00H	0	10 Seconds			Seconds				Seconds	00–59
01H	0	10 Minutes			Minutes				Minutes	00–59
02H	0	12/$\overline{24}$	$\overline{\text{AM}}$/PM 10 Hour	10 Hour	Hour				Hours	1–12 + $\overline{\text{AM}}$/PM 00–23
03H	0	0	0	0	0	Day			Day	1–7
04H	0	0	10 Date		Date				Date	00–31
05H	Century	0	0	10 Month	Month				Month/ Century	01–12 + Century
06H	10 Year				Year				Year	00–99
07H	A1M1	10 Seconds			Seconds				Alarm 1 Seconds	00–59
08H	A1M2	10 Minutes			Minutes				Alarm 1 Minutes	00–59
09H	A1M3	12/$\overline{24}$	$\overline{\text{AM}}$/PM 10 Hour	10 Hour	Hour				Alarm 1 Hours	1–12 + $\overline{\text{AM}}$/PM 00–23
0AH	A1M4	DY/$\overline{\text{DT}}$	10 Date		Day				Alarm 1 Day	1–7
					Date				Alarm 1 Date	1–31
0BH	A2M2	10 Minutes			Minutes				Alarm 2 Minutes	00–59
0CH	A2M3	12/$\overline{24}$	$\overline{\text{AM}}$/PM 10 Hour	10 Hour	Hour				Alarm 2 Hours	1–12 + $\overline{\text{AM}}$/PM 00–23
0DH	A2M4	DY/$\overline{\text{DT}}$	10 Date		Day				Alarm 2 Day	1–7
					Date				Alarm 2 Date	1–31
0EH	$\overline{\text{EOSC}}$	BBSQW	CONV	RS2	RS1	INTCN	A2IE	A1IE	Control	—
0FH	OSF	0	0	0	EN32kHz	BSY	A2F	A1F	Control/Status	—
10H	SIGN	DATA	DATA	DATA	DATA	DATA	DATA	DATA	Aging Offset	—
11H	SIGN	DATA	DATA	DATA	DATA	DATA	DATA	DATA	MSB of Temp	—
12H	DATA	DATA	0	0	0	0	0	0	LSB of Temp	—

- **Temperature Compensation:** Higher-end RTCs may include temperature-compensated crystal oscillators (TCXOs) to maintain accuracy across a range of temperatures.
- **Ease of Integration:** RTCs usually have straightforward interfaces and communication protocols, making them relatively easy to integrate into microcontroller-based systems. This aligns well with the project's goal of combining a real-time clock, LCD display, and sensor.
- **Reliability**: RTCs are designed for reliable timekeeping, and their accuracy is generally better than relying solely on a microcontroller's internal clock.

LCD: GJD 1602IIC liquid crystal display:

To display the result of the code. Compatible with the devices we used in terms of wiring communication and power consumption.

Switch:

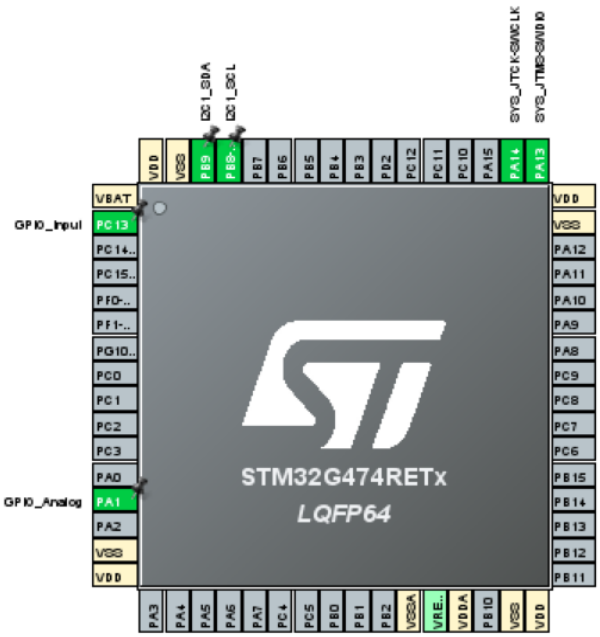
simple switch that is used to change the output of the LCD. Depending on the condition of the code if it is set, we get the value of the values of the temperature and humidity else we get the time and date from RTC.

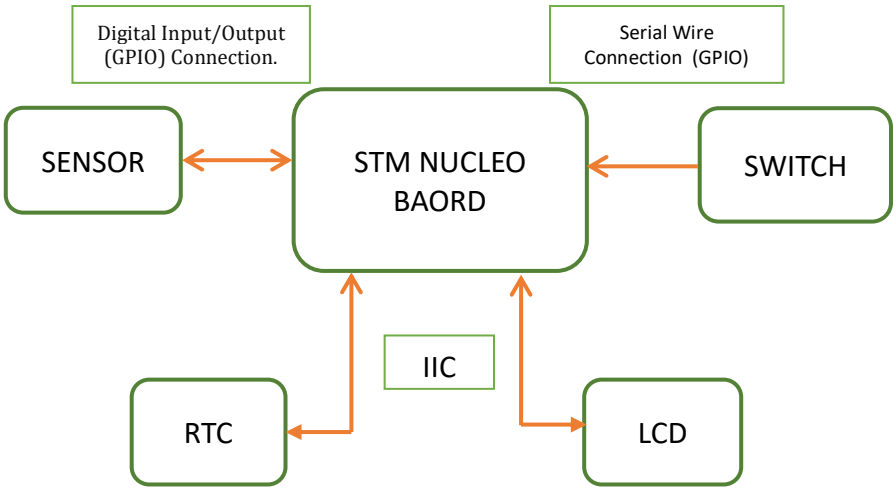
Peripherals initialized in STM32CubeMX:

I2C peripherals:
LDC, RTC ➔ I2C1_SDA ➔ PB_9
I2C1_SCL ➔ PB_8

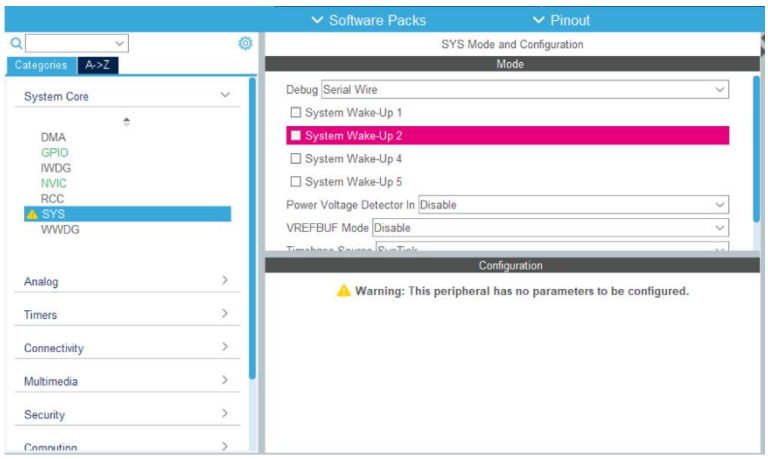
GPIO:
SWITCH ➔ GPIO_INPUT ➔ PC_13
Sensor ➔ GPIO_ANALOG ➔ PA_1

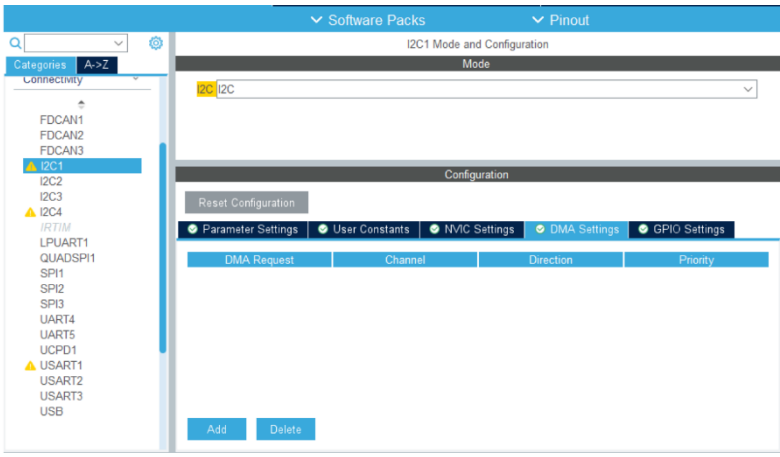
the ARM Cortex debug interface:
the clock signal line ➔ SYS_JTCK-SWCLK ➔ PA_13
the bidirectional data line ➔ SYS_JTMS-SWDIO ➔PA_14

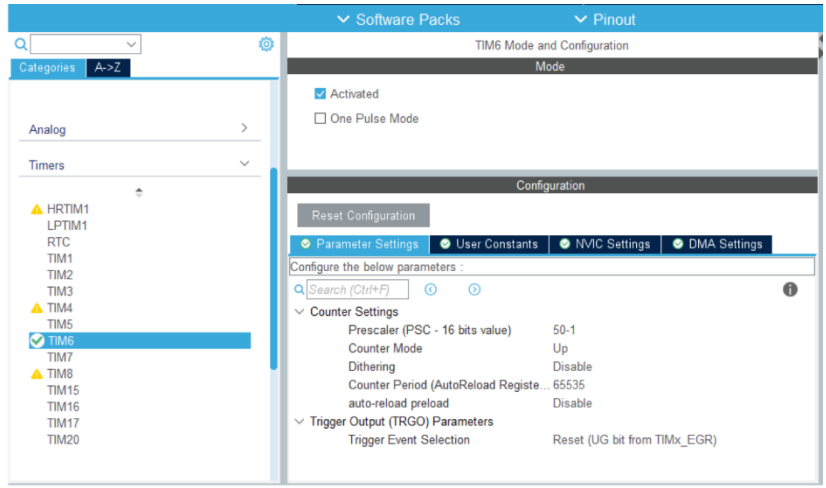




STM32CUBEMX CONFIGURATION:





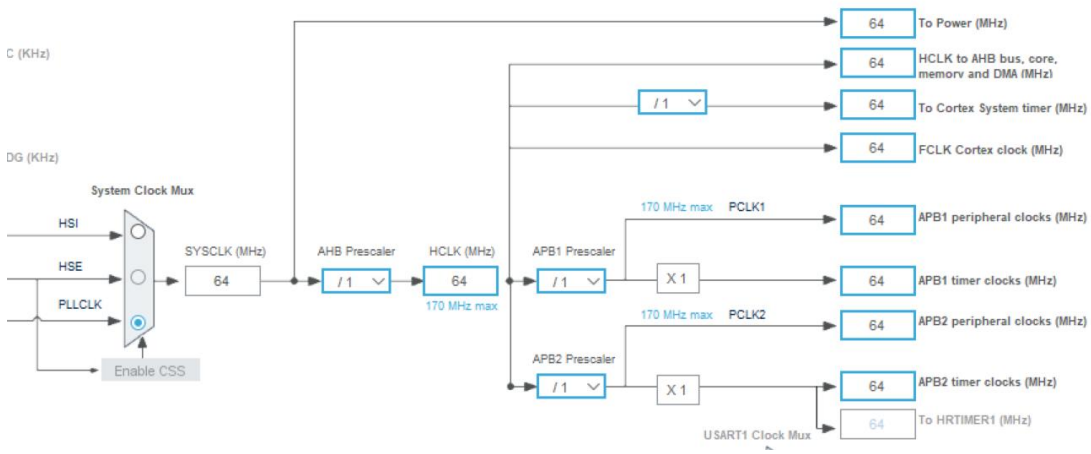


Timer 06 ➔ for creating delay in microseconds.
Frequency of oscillator on board ➔ Left at 8 Mhz.
Controller ➔ set to run at 64 Mhz.
APB1 timer clocks will also run at 64 MHz, and it is where my timer is connecting ➔ This ensures synchronization between the timer used for delays and the overall system clock.

Overall System Performance: By running the controller and APB1 timer clocks at 64 MHz, the processing speed of the STM Nucleo G474RET3

microcontroller is increased. This results in improved overall system performance, allowing for faster.

execution of tasks, reduced latency in responding to user inputs, and smoother operation of the entire system.



Code written in CubeIDE:

```
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include "i2c-lcd.h" // to use LCD library
#include "stdio.h"  // for sprintf function
/* USER CODE END Includes */
/* USER CODE BEGIN 0 */
// DHT11 SENSOR//
void delay (uint16_t time)
{

/*delay in microseconds >> set counter to 0 then wait for counter to reach the wanted value */

        __HAL_TIM_SET_COUNTER(&htim6, 0);
   while ((__HAL_TIM_GET_COUNTER(&htim6))<time);
}



void Display_Temp(float Temp)
{
        char str[20] = {0};
```

```
        lcd_put_cur(0,0);
        sprintf(str, "TEMP:- %.2f", Temp);
        lcd_send_string(str);
        lcd_send_data('C');
}

void Display_RH(float RH)
{
        char str[20] = {0};

        lcd_put_cur(1,0);
        sprintf(str, "RH:- %.2f", RH);
        lcd_send_string(str);
        lcd_send_data('%');
}


uint8_t Rh_byte1, Rh_byte2, Temp_byte1, Temp_byte2;
uint16_t SUM, RH, TEMP;

float Temperature;
float Humidity;
float Presence;

// set pin to input or output based on the requirements

void Set_Pin_Output(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
{
        GPIO_InitTypeDef GPIO_InitStruct = {0};

        GPIO_InitStruct.Pin = GPIO_Pin;
        GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
        GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;

        HAL_GPIO_Init(GPIOx, &GPIO_InitStruct);
}



void Set_Pin_Input(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
{
        GPIO_InitTypeDef GPIO_InitStruct = {0};

        GPIO_InitStruct.Pin = GPIO_Pin;
        GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
        GPIO_InitStruct.Pull = GPIO_NOPULL;

        HAL_GPIO_Init(GPIOx, &GPIO_InitStruct);
```

```
}


// define pin and port for the sensor//
#define DHT11_PORT GPIOA
#define DHT11_PIN GPIO_PIN_1


void DHT11_Start (void) // all from datasheet
{
        Set_Pin_Output (DHT11_PORT, DHT11_PIN);         // set the pin as output
        HAL_GPIO_WritePin (DHT11_PORT, DHT11_PIN, 0);   // pull the pin low
        delay (18000);                          // wait for 18ms
        HAL_GPIO_WritePin (DHT11_PORT, DHT11_PIN, 1);   // pull the pin high
   delay (20);
   Set_Pin_Input(DHT11_PORT, DHT11_PIN);          // set as input
}



uint8_t DHT_Check_Response (void)
{
        uint8_t Response = 0;
        delay (40);

        if (!(HAL_GPIO_ReadPin (DHT11_PORT, DHT11_PIN)))
        {
                delay (80);

                if ((HAL_GPIO_ReadPin (DHT11_PORT, DHT11_PIN))) Response = 1;

                else Response = -1;
        }

        while ((HAL_GPIO_ReadPin (DHT11_PORT, DHT11_PIN)));   // wait for the pin to go
low
   return Response;
}



uint8_t DHT_Read (void)
{
        uint8_t i,j;

        for (j=0;j<8;j++)
```

```
        {
                while (!(HAL_GPIO_ReadPin (DHT11_PORT, DHT11_PIN)));   // wait for the pin
to go high
                delay (40);                                // wait for 40 us

                if (!(HAL_GPIO_ReadPin (DHT11_PORT, DHT11_PIN)))       // if the pin is low
                {
                        i&= ~(1<<(7-j));                           // write 0
                }

                else i|= (1<<(7-j));                           // if the pin is high, write 1
                while ((HAL_GPIO_ReadPin (DHT11_PORT, DHT11_PIN)));    // wait for the pin
to go low
        }

        return i;
}



//DS3231 RTC//
#define DS3231_ADDRESS 0xD0  // address of RTC mentioned in it's datasheet. it is needed for
the IIC communication


// the data written in the DS3231 are in BCD format so we need some conversion
// Convert normal decimal numbers to binary coded decimal
uint8_t decToBcd(int val)
{
   return (uint8_t)((val/10*16) + (val%10));
}


// Convert binary coded decimal to normal decimal numbers
int bcdToDec(uint8_t val)
{
   return (int)((val/16*10) + (val%16));
}


// structure to save time and date that we are going to read from the RTC
typedef struct {
        uint8_t seconds;
        uint8_t minutes;
        uint8_t hour;
        uint8_t dayofweek;
        uint8_t dayofmonth;
```

```
        uint8_t month;
        uint8_t year;
        uint8_t humidity;
        uint8_t temperature;

} TIME;

TIME time;



// function to set the time

void Set_Time (uint8_t sec, uint8_t min, uint8_t hour, uint8_t dow, uint8_t dom, uint8_t month,
uint8_t year) // our parameters
{
        uint8_t set_time[7];      // an array that stores seven bytes of data (our parameters)

        // data must follow this order depending on the datasheet
        // we first convert data to BCD then we store them in a BCD format
        set_time[0] = decToBcd(sec);
        set_time[1] = decToBcd(min);
        set_time[2] = decToBcd(hour);
        set_time[3] = decToBcd(dow);
        set_time[4] = decToBcd(dom);
        set_time[5] = decToBcd(month);
        set_time[6] = decToBcd(year);

        // write the values to the registers using the fact that address of the registers in the module
increments automatically with every operation( This operation is called Multi-Write)
        HAL_I2C_Mem_Write(&hi2c1, DS3231_ADDRESS, 0x00, 1, set_time, 7, 1000); // use of
memory write function to write data in specific location in the memory of the device
 /* Write an amount of data in blocking mode to a specific memory address
  *hi2c Pointer to a I2C_HandleTypeDef structure that contains
  *             the configuration information for the specified I2C.
  * Device_Address Target device address: The device 7 bits address value
  *        in datasheet must be shifted to the left before calling the interface
  * MemAddress Internal memory address
  * MemAddSize Size of internal memory address
  * pData Pointer to data buffer
  * Size Amount of data to be sent
  * Timeout duration
  */
}
```

```
HAL_StatusTypeDef HAL_I2C_Mem_Write(I2C_HandleTypeDef *hi2c, uint16_t DevAddress,
uint16_t MemAddress, uint16_t MemAddSize, uint8_t *pData, uint16_t Size, uint32_t Timeout);

// to read time from RTC and save it into the time structure
void Get_Time(void)
{
        uint8_t get_time[7]; // we create an array to store bcd value that we receive from RTC

        HAL_I2C_Mem_Read(&hi2c1, DS3231_ADDRESS, 0x00, 1, get_time, 7, 1000); // to read
data from memory address (0x00) directly, and we are reading 7 bytes of data from this address, so
1 byte will be read from the each incremented address
        time.seconds = bcdToDec(get_time[0]);
        time.minutes = bcdToDec(get_time[1]);
        time.hour = bcdToDec(get_time[2]);
        time.dayofweek = bcdToDec(get_time[3]);
        time.dayofmonth = bcdToDec(get_time[4]);
        time.month = bcdToDec(get_time[5]);
        time.year = bcdToDec(get_time[6]);
}
// create buffer to store the equivalent characters to display them on LCD
char buffer[15];



int _write(int file, char *ptr, int len){
        int i=0;
        for (i=0; i<len; i++){
                ITM_SendChar((*ptr++));
        } return len;
}
char variable=0;
/* USER CODE END 0 */
/* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */
/* USER CODE BEGIN 2 */

  HAL_TIM_Base_Start(&htim6);
  lcd_init();                // initialize  the LCD
  lcd_send_string("INITIALISING>>>");
  HAL_Delay(2000);
  lcd_clear();
```

```
  //set the time once in the main function, and than we can always read the time in the while loop.
  //Set_Time( 03, 50, 10, 5, 01, 05, 24);

  /* USER CODE END 2 */
while (1)
  {
   /* USER CODE END WHILE */

   /* USER CODE BEGIN 3 */

          GPIO_PinState g_sw = HAL_GPIO_ReadPin( GPIOC, GPIO_PIN_13 ); // reads the state
of a GPIO pin connected to a switch

          if ( g_sw ==GPIO_PIN_RESET ) //switch is pressed
          {
        Get_Time ( ); // read time

              sprintf (buffer, "%02d:%02d:%02d", time.hour, time.minutes, time.seconds ); // display
time
                    lcd_put_cur ( 0,0 );
                    lcd_send_string(buffer);

                    sprintf (buffer, "%02d-%02d-20%02d", time.dayofmonth, time.month, time.year );
// display date
                    lcd_put_cur ( 1, 0 );
                    lcd_send_string(buffer);
                    HAL_Delay ( 500 );
          }

          else // switch is not pressed
          {

              Display_Temp ( Temperature );
              Display_RH ( Humidity );
        DHT11_Start ();

                    Presence = DHT_Check_Response ();

                    Rh_byte1 = DHT_Read ();
                    Rh_byte2 = DHT_Read ();
                    Temp_byte1 = DHT_Read ();
                    Temp_byte2 = DHT_Read ();
        SUM = DHT_Read ();

        RH= Rh_byte1;
              TEMP= Temp_byte1;
```

```
        Temperature = (float) TEMP;
        Humidity = (float) RH;

        if (SUM == (Rh_byte1+Rh_byte2+Temp_byte1+Temp_byte2))
            {printf ( "tutto bene", variable+1 );
            }
        else {printf ( "qualcosa non va", variable );
            }
        HAL_Delay(2000);
        lcd_clear();
    }


}
 /* USER CODE END 3 */
}
```

"i2c-lcd.c" folder:

```
/*This code provides a set of functions to control an LCD display using the I2C protocol.
 * The code initializes the LCD, sends commands, sends data, clears the display, sets the cursor
position,
 *  and sends strings to the display.
 */

#include "i2c-lcd.h" // header file for the LCD
extern I2C_HandleTypeDef hi2c1;

#define SLAVE_ADDRESS_LCD 0x7E //defines the I2C address of the LCD.



void lcd_send_cmd (char cmd) // sends a command to the LCD.
{
   char data_u, data_l; // hold the upper and lower nibbles (4 bits) of the command byte,
respectively.

        uint8_t data_t[4]; // hold the data to be transmitted over I2C.

        data_u = (cmd&0xf0);  //extracts the upper nibble of cmd by performing a bitwise AND
with 0xf0.
        data_l = ((cmd<<4)&0xf0); // extracts the lower nibble of cmd by left-shifting cmd by 4 bits
and then performing a bitwise AND with 0xf0.

        //Preparing Data to be Transmitted
        data_t[0] = data_u|0x0C;  //en=1, rs=0  // combines data_u (upper nibble) with control bits
to set the enable bit (en=1) and register select bit (rs=0). The resulting byte is sent to the LCD to
latch the upper nibble.
        data_t[1] = data_u|0x08;  //en=0, rs=0  // This byte is sent to the LCD to disable the enable
line after latching the upper nibble.
        data_t[2] = data_l|0x0C;  //en=1, rs=0
        data_t[3] = data_l|0x08;  //en=0, rs=0

        //Transmitting Data via I2C
        HAL_I2C_Master_Transmit(&hi2c1, SLAVE_ADDRESS_LCD,(uint8_t *) data_t, 4, 100);


void lcd_send_data (char data) // sends data to the LCD
{
        char data_u, data_l;

        uint8_t data_t[4];

        data_u = (data&0xf0);
        data_l = ((data<<4)&0xf0);
```

```
        data_t[0] = data_u|0x0D;  //en=1, rs=1
        data_t[1] = data_u|0x09;  //en=0, rs=1
        data_t[2] = data_l|0x0D;  //en=1, rs=1
        data_t[3] = data_l|0x09;  //en=0, rs=1

        HAL_I2C_Master_Transmit(&hi2c1, SLAVE_ADDRESS_LCD,(uint8_t *)data_t, 4, 100);
}

void lcd_clear(void) //clears the LCD by moving the cursor to the start and writing 70 spaces
(enough to clear a typical 2x16 or 4x20 LCD).
{
        lcd_send_cmd(0x80);

        for(int i=0; i<70; i++)
        {
                lcd_send_data(' ');
        }
}

void lcd_put_cur(int row, int col) //sets the cursor position on the LCD. It adjusts the column value
based on the row and sends the appropriate command.
{
    switch(row)
    {
      case 0:
        col |= 0x80;
        break;

      case 1:
        col |= 0xC0;
        break;
    }
    lcd_send_cmd(col);



void lcd_init(void) //  initializes the LCD in 4-bit mode
{
        // 4 bit initialisation
        HAL_Delay(50);          // wait for >40ms
        lcd_send_cmd(0x30);
        HAL_Delay(5);           // wait for >4.1ms
        lcd_send_cmd(0x30);
        HAL_Delay(1);           // wait for >100us
        lcd_send_cmd(0x30);
        HAL_Delay(10);
        lcd_send_cmd(0x20);     // 4bit mode
        HAL_Delay(10);
```

```
  // dislay initialisation
        lcd_send_cmd (0x28);    // Function set --> DL=0 (4 bit mode), N = 1 (2 line display) F = 0
(5x8 characters)
        HAL_Delay(1);
        lcd_send_cmd (0x08);    //Display on/off control --> D=0,C=0, B=0  ---> display off
        HAL_Delay(1);
        lcd_send_cmd (0x01);    // clear display
        HAL_Delay(1);
        HAL_Delay(1);
        lcd_send_cmd (0x06);    //Entry mode set --> I/D = 1 (increment cursor) & S = 0 (no shift)
        HAL_Delay(1);
        lcd_send_cmd (0x0C);    //Display on/off control --> D = 1, C and B = 0. (Cursor and blink,
last two bits)


//sends a null-terminated string to the LCD by repeatedly calling lcd_send_data for each character
in the string.
void lcd_send_string (char *str)
{
        while (*str) lcd_send_data (*str++);
}
```

"i2c-lcd.h" folder:

```
/*This code defines function prototypes for interacting with an LCD screen
using the STM32G4 microcontroller. The functions handle various tasks*/
#include "stm32g4xx_hal.h"

void lcd_init (void);   // initialize lcd

void lcd_send_cmd (char cmd);  // send command to the lcd

void lcd_send_data (char data);  // send data to the lcd

void lcd_send_string (char *str);  // send string to the lcd

void lcd_put_cur(int row, int col);  // put cursor at the entered position row (0 or 1), col (0-15);

void lcd_clear (void);
```